

Retail Microservices Platform on AWS

A blueprint to ship changes safely, stay online on bad days, and secure everything by default

A working project any team can adopt



The shop

12 services on Amazon EKS



The delivery line

GitHub Actions · Terraform · Helm



The safety nets

rollback · alerts · scans · tests

The problem every online business has

Three risks that cost money, whatever the company size



Risky releases

Manual deployments break things, and going back takes hours.



Costly outages

A crash or a traffic peak takes the shop offline, and customers leave.



Security last

Passwords in code, admin access for everyone, problems found too late.



What if every change were tested, approved and reversible, and the system repaired itself?

The goal: four promises

What the platform guarantees, and the tools behind each promise



Ship safely

Every change is tested, approved by a person, and can be undone in one click.

How: [GitHub Actions](#) • [Helm rollback](#)



Stay online

Broken parts are replaced automatically, and capacity grows at rush hour.

How: [Kubernetes \(EKS\)](#) • [autoscaling](#) • [queues](#)



Secure by default

No stored passwords, minimal access for each part, code and images scanned.

How: [OIDC](#) • [IAM roles](#) • [CodeQL](#) • [ECR scan](#)



See everything

Live dashboards, and alerts on the team's phones before customers notice.

How: [Prometheus](#) • [Grafana](#) • [CloudWatch](#) • [Slack](#)

How it works

One order, from a click to the confirmation email



Customer

orders on the website



Website

secure connection (HTTPS)



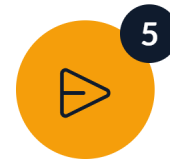
12 services

cart, payment,
shipping, stock...



Waiting line

orders wait their turn,
none lost



Email + Slack

customer and team are told



The store's memory

Products and orders, carts and stock levels, in managed databases: private, encrypted and backed up.

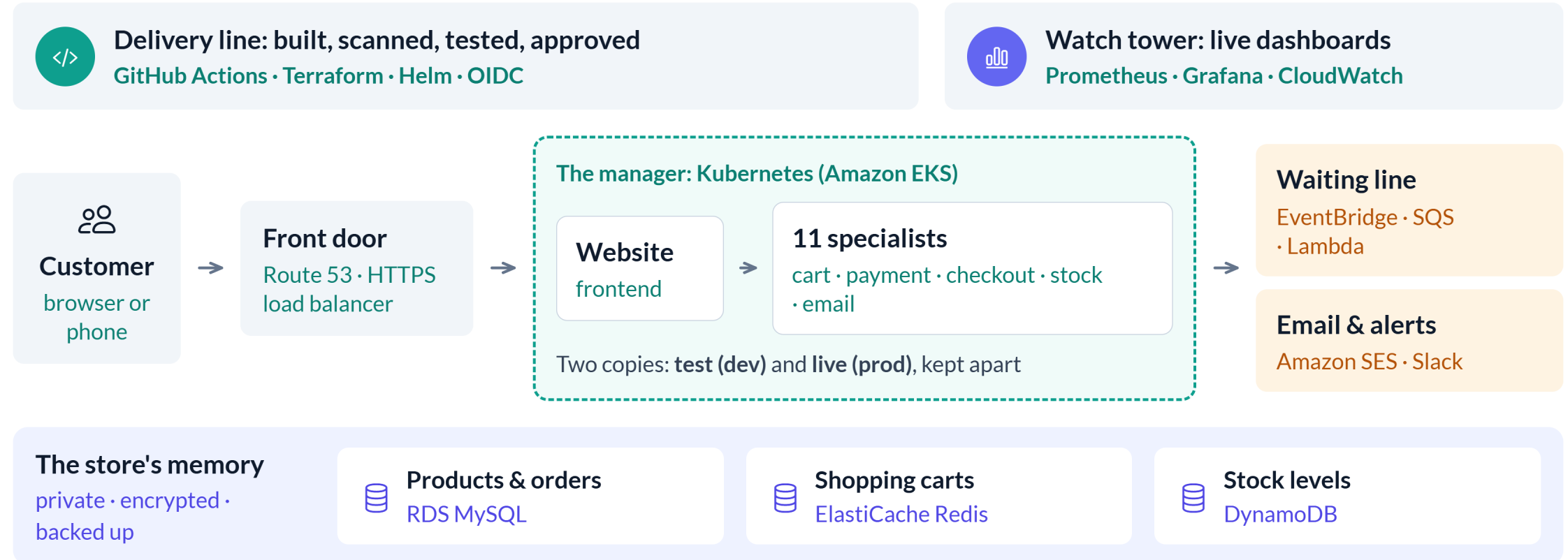


A manager watches everything

Kubernetes replaces what breaks, adds capacity at rush hour, and alerts the team if orders get stuck.

Architecture at a glance

What each part does in **bold**, the technology behind it in teal



Live demo

About two minutes: one real order, end to end

- 1 Open the shop**
shop.sebngembou-cloud.click, with the padlock
- 2 Buy something**
check out with my own email address
- 3 Watch my phone**
the confirmation email arrives in seconds
- 4 Check Slack**
the team sees "order confirmed"

FROM A REAL ORDER

Your order 8cb7947b is confirmed  Boîte de réception x



orders@sebngembou-cloud.click

22:29 (il y a 0 minute) ☆

À moi ▾

Thank you for your order!

All your items are reserved and your order is being prepared for shipping.

Order ID: 8cb7947b-c064-11f1-a814-aafaa043e5fd



Boutique-Microservice APP 10:29 PM

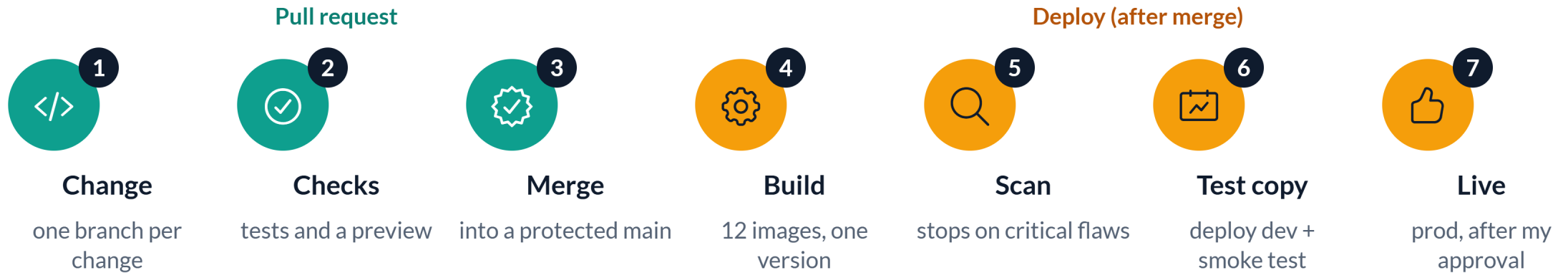


Order **8cb7947b** confirmed [dev]: order reserved, ready to ship.

Backup if the Wi-Fi fails

From code to customers

Every change follows the same path, with a check at every step



Automatic rollback

A failed deploy brings the previous version back by itself.



One-click undo

Any earlier version can be redeployed, with nothing rebuilt.



Slack everywhere

Deploys, approvals, alarms and orders, each in its own channel.

Proven under pressure

I broke it on purpose, three different ways



A part crashes

5 s

to replace it automatically, with 0 failed visits

Chaos: pod failure (checkoutservice)

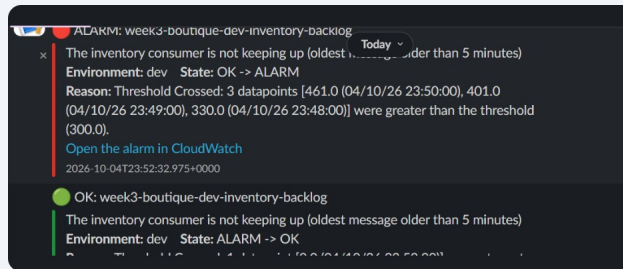
- Pods ready before: 1. Deleting checkoutservice-58f9f9459c-19b6q at 02:57:42 UTC.
-  New pod checkoutservice-58f9f9459c-1vfxm Ready after 5 s (Kubernetes recreated it, nobody had to act).
- Shop home page during the test: 12 OK, 0 errors (one check every 2 s).



A station stops 8 min

0

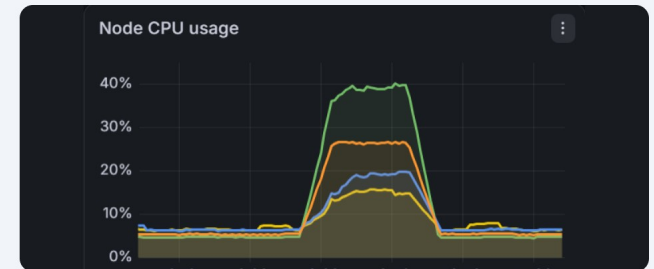
orders lost; the alarm went red, then green



800 shoppers at once

~10x

faster home page with automatic scaling



How to adopt it in your company

Start small: each step pays off on its own, for any application

1

Foundations

Describe the infrastructure as code and remove stored passwords.

Terraform • OIDC

2

Safe delivery

Pull requests, automatic checks, a test copy, approval and one-click undo.

GitHub Actions • Helm

3

Resilience

Run on Kubernetes, add waiting lines and automatic scaling.

EKS • SQS • autoscaling

4

Confidence

Dashboards, alerts and scans, then break it on purpose.

Grafana • CodeQL • chaos tests

~\$10 a day

for a test and a live copy, switched off at night

All code

every piece lives in one repository, ready to reuse

Any app

the delivery line and safety nets don't depend on this shop

Three things to take away

- 1 Automate the path to production: tested, approved, reversible.
- 2 Design for failure: break it on purpose, before customers do.
- 3 Security and visibility from day one, not after an incident.

Thank you

Questions?

Sebastien Ngembou

github.com/ngems1/microservice